HOPE IA R = f (Q)

assistantIA

Exportation complète du code source

dimanche 31 mai 2026 à 14:16:52
4 fichier(s)
Généré par kojo

voir plus

templates\index.html

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Hope Assistant · structuré</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-
beta3/css/all.min.css">
    <link rel="stylesheet" href="/static/style.css">
    <!-- Marked.js pour le rendu Markdown -->
    <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>

</head>
<body>
<div class="sidebar">
    <button class="new-chat" id="newBtn"><i class="fas fa-plus"></i> Nouveau chat</button>
    <div class="conv-list" id="convList"></div>
</div>
<div class="main">
    <div class="header">
        <h2 id="title">Conversation</h2>
        <button class="clear-btn" id="clearBtn"><i class="fas fa-trash-alt"></i>
Effacer</button>
    </div>
    <div class="chat-area" id="chatArea"></div>
    <div class="input-wrapper">
        <div class="input-container">
            <textarea id="msgInput" rows="1" placeholder="Écrivez votre question...
(Entrée)"></textarea>
            <button class="send" id="sendBtn"><i class="fas fa-paper-plane"></i></button>
        </div>
        <div class="footer">Hope Assistant, verifiez les réponses car il peut se
tromper</div>
    </div>
</div>

<script>
    // ---------- données ----------
    let conversations = [];
    let currentId = null;

    // Configuration de marked (optionnel)
    marked.setOptions({
        breaks: true,      // conversion des retours à la ligne en <br>
        gfm: true          // GitHub Flavored Markdown
```

```
    });

    function escapeHtml(str) {
        return str.replace(/[&<>]/g, function(m) {
            if (m === '&') return '&amp;';
            if (m === '<') return '&lt;';
            if (m === '>') return '&gt;';
            return m;
        });
    }

    // Convertir le markdown en HTML (synchrone)
    function renderMarkdown(text) {
        try {
            return marked.parse(text, { async: false });
        } catch(e) {
            return escapeHtml(text).replace(/\n/g, '<br>');
        }
    }

    function save() { localStorage.setItem('hope_simple', JSON.stringify(conversations)); }
    function load() {
        const raw = localStorage.getItem('hope_simple');
        if (raw) conversations = JSON.parse(raw);
        if (!conversations.length) {
            conversations = [{ id: Date.now(), title: "Nouveau chat", messages: [{ role:
"bot", text: "Bonjour ! Je suis Hope Assistant. Posez vos questions." }] }];
            save();
        }
        currentId = conversations[0].id;
        renderSidebar();
        renderChat();
    }

    function renderSidebar() {
        const container = document.getElementById('convList');
        container.innerHTML = '';
        conversations.forEach(c => {
            const div = document.createElement('div');
            div.className = `conv ${c.id === currentId ? 'active' : ''}`;
            div.innerHTML = `<span>${escapeHtml(c.title)}</span><i class="fas fa-trash-alt
del-conv"></i>`;
            div.addEventListener('click', (e) => { if (!e.target.classList.contains('del-
conv')) { currentId = c.id; renderSidebar(); renderChat(); } });
            div.querySelector('.del-conv').addEventListener('click', (e) => {
e.stopPropagation(); deleteConv(c.id); });
            container.appendChild(div);
        });
    }
```

```
    function deleteConv(id) {
        if (conversations.length === 1) return alert("Au moins une conversation requise");
        conversations = conversations.filter(c => c.id !== id);
        if (currentId === id) currentId = conversations[0].id;
        save();
        renderSidebar();
        renderChat();
    }

    function newConversation() {
        const newId = Date.now();
        conversations.unshift({ id: newId, title: "Nouveau chat", messages: [] });
        currentId = newId;
        save();
        renderSidebar();
        renderChat();
    }

    function updateTitle(conv) {
        const firstUser = conv.messages.find(m => m.role === 'user');
        conv.title = firstUser ? (firstUser.text.slice(0, 25) + (firstUser.text.length > 25
? '…' : '')) : "Nouveau chat";
        save();
        renderSidebar();
        document.getElementById('title').innerText = conv.title;
    }

    function renderChat() {
        const area = document.getElementById('chatArea');
        const conv = conversations.find(c => c.id === currentId);
        if (!conv) return;
        area.innerHTML = '';
        if (conv.messages.length === 0) {
            area.innerHTML = '<div class="empty"><i class="fas fa-comment-dots"></i>
<p>Comment puis-je vous aider ?</p></div>';
            return;
        }
        conv.messages.forEach((msg, idx) => {
            const div = document.createElement('div');
            div.className = `message ${msg.role}`;
            const avatar = `<div class="avatar"><i class="fas ${msg.role === 'user' ? 'fa-
user' : 'fa-robot'}"></i></div>`;
            let bubbleContent = '';
            if (msg.role === 'user') {
                bubbleContent = `<div class="bubble"
id="bubble-${idx}">${escapeHtml(msg.text).replace(/\n/g, '<br>')}<i class="fas fa-pen"
style="font-size:0.7rem; margin-left:8px; cursor:pointer;" onclick="editMessage(${idx})">
</i></div>`;
```

```
            } else {
                // Pour le bot : conversion Markdown -> HTML
                const htmlContent = renderMarkdown(msg.text);
                bubbleContent = `<div class="bubble" id="bubble-${idx}">${htmlContent}
</div>`;
            }
            div.innerHTML = avatar + bubbleContent;
            area.appendChild(div);
        });
        area.scrollTop = area.scrollHeight;
        document.getElementById('title').innerText = conv.title;
    }

    // Édition en ligne (inchangée)
    window.editMessage = function(msgIndex) {
        const conv = conversations.find(c => c.id === currentId);
        if (!conv) return;
        const oldText = conv.messages[msgIndex].text;
        const bubbleDiv = document.getElementById(`bubble-${msgIndex}`);
        if (!bubbleDiv) return;
        bubbleDiv.innerHTML = '';
        const textarea = document.createElement('textarea');
        textarea.value = oldText;
        textarea.className = 'edit-input';
        textarea.rows = 2;
        const saveBtn = document.createElement('button');
        saveBtn.textContent = '✓';
        saveBtn.style.marginLeft = '8px';
        saveBtn.style.padding = '4px 8px';
        saveBtn.style.borderRadius = '20px';
        saveBtn.style.border = 'none';
        saveBtn.style.background = '#2563eb';
        saveBtn.style.color = 'white';
        saveBtn.style.cursor = 'pointer';
        const cancelBtn = document.createElement('button');
        cancelBtn.textContent = '✗';
        cancelBtn.style.marginLeft = '6px';
        cancelBtn.style.padding = '4px 8px';
        cancelBtn.style.borderRadius = '20px';
        cancelBtn.style.border = 'none';
        cancelBtn.style.background = '#94a3b8';
        cancelBtn.style.cursor = 'pointer';
        bubbleDiv.appendChild(textarea);
        bubbleDiv.appendChild(saveBtn);
        bubbleDiv.appendChild(cancelBtn);
        textarea.focus();

        const saveEdit = () => {
            const newText = textarea.value.trim();
```

```
            if (newText !== '') {
                conv.messages[msgIndex].text = newText;
                save();
                updateTitle(conv);
                renderChat();
            } else {
                renderChat();
            }
        };
        saveBtn.onclick = saveEdit;
        cancelBtn.onclick = () => renderChat();
        textarea.addEventListener('keypress', (e) => {
            if (e.key === 'Enter' && !e.shiftKey) {
                e.preventDefault();
                saveEdit();
            }
        });
    };

    async function sendMessage() {
        const input = document.getElementById('msgInput');
        let q = input.value.trim();
        if (!q) return;
        const conv = conversations.find(c => c.id === currentId);
        conv.messages.push({ role: 'user', text: q });
        updateTitle(conv);
        renderChat();
        input.value = '';

        const area = document.getElementById('chatArea');
        const typingDiv = document.createElement('div');
        typingDiv.className = 'message bot';
        typingDiv.innerHTML = `<div class="avatar"><i class="fas fa-robot"></i></div><div
class="typing"><span></span><span></span><span></span></div>`;
        area.appendChild(typingDiv);
        area.scrollTop = area.scrollHeight;

        try {
            const res = await fetch('/chat', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ question: q })
            });
            const data = await res.json();
            typingDiv.remove();
            conv.messages.push({ role: 'bot', text: data.reponse });
            save();
            renderChat();
        } catch(e) {
```

```
            typingDiv.remove();
            conv.messages.push({ role: 'bot', text: "Erreur de connexion au serveur." });
            save();
            renderChat();
        }
    }

    // événements
    document.getElementById('sendBtn').onclick = sendMessage;
    document.getElementById('msgInput').addEventListener('keypress', (e) => { if (e.key ===
'Enter' && !e.shiftKey) { e.preventDefault(); sendMessage(); } });
    document.getElementById('clearBtn').onclick = () => {
        if (confirm('Effacer tous les messages de cette conversation ?')) {
            const conv = conversations.find(c => c.id === currentId);
            conv.messages = [];
            updateTitle(conv);
            save();
            renderChat();
        }
    };
    document.getElementById('newBtn').onclick = newConversation;

    load();
</script>
</body>
</html>
```

static\style.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    background: #f5f7fb;
    font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
    height: 100vh;
    display: flex;
    overflow: hidden;
}

/* Sidebar */
.sidebar {
    width: 260px;
    background: #fff;
    border-right: 1px solid #e2e8f0;
    display: flex;
    flex-direction: column;
    padding: 20px 12px;
    gap: 16px;
}

.new-chat {
    background: #f1f5f9;
    border: none;
    border-radius: 30px;
    padding: 8px 12px;
    font-weight: 500;
    cursor: pointer;
    display: flex;
    align-items: center;
    gap: 8px;
    justify-content: center;
}

.new-chat:hover {
    background: #e2e8f0;
}

.conv-list {
    flex: 1;
    overflow-y: auto;
```

```
    display: flex;
    flex-direction: column;
    gap: 4px;
}

.conv {
    padding: 8px 12px;
    border-radius: 10px;
    cursor: pointer;
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.85rem;
}

.conv.active {
    background: #eef2ff;
    font-weight: 500;
}

.conv span {
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
    flex: 1;
}

.del-conv {
    opacity: 0;
    color: #94a3b8;
    cursor: pointer;
}

.conv:hover .del-conv {
    opacity: 1;
}

/* Main */
.main {
    flex: 1;
    display: flex;
    flex-direction: column;
    background: #fff;
}

.header {
    padding: 12px 20px;
    border-bottom: 1px solid #edf2f7;
    display: flex;
```

```
    justify-content: space-between;
    align-items: center;
}

.header h2 {
    font-size: 1.2rem;
    font-weight: 500;
}

.clear-btn {
    background: none;
    border: none;
    color: #64748b;
    cursor: pointer;
}

.chat-area {
    flex: 1;
    overflow-y: auto;
    padding: 20px;
    display: flex;
    flex-direction: column;
    gap: 16px;
}

.empty {
    flex: 1;
    display: flex;
    align-items: center;
    justify-content: center;
    color: #94a3b8;
    text-align: center;
}

.message {
    display: flex;
    gap: 10px;
    max-width: 80%;
}

.user {
    align-self: flex-end;
    flex-direction: row-reverse;
}

.avatar {
    width: 30px;
    height: 30px;
    background: #e2e8f0;
```

```
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 0.8rem;
}

.user .avatar {
    background: #2563eb;
    color: white;
}

.bubble {
    background: #f1f5f9;
    padding: 8px 14px;
    border-radius: 18px;
    font-size: 0.9rem;
    line-height: 1.4;
    word-break: break-word;
}

.user .bubble {
    background: #2563eb;
    color: white;
}

/* Styles pour le rendu Markdown (titres, code, listes) */
.bubble h1,
.bubble h2,
.bubble h3 {
    margin: 12px 0 6px;
    font-weight: 600;
}

.bubble h2 {
    font-size: 1.2rem;
}

.bubble h3 {
    font-size: 1rem;
}

.bubble p {
    margin: 6px 0;
}

.bubble ul,
.bubble ol {
    margin: 6px 0 6px 20px;
```

```
}

.bubble li {
    margin: 2px 0;
}

.bubble pre {
    background: #1e293b;
    color: #e2e8f0;
    padding: 10px;
    border-radius: 12px;
    overflow-x: auto;
    font-family: monospace;
    font-size: 0.8rem;
    margin: 10px 0;
}

.bubble code {
    background: #eef2ff;
    padding: 2px 6px;
    border-radius: 8px;
    font-family: monospace;
    font-size: 0.85rem;
}

.bubble pre code {
    background: none;
    padding: 0;
}

.edit-input {
    background: white;
    border: 1px solid #cbd5e1;
    border-radius: 18px;
    padding: 6px 12px;
    font-family: inherit;
    font-size: 0.9rem;
    width: 100%;
    outline: none;
}

.edit-input:focus {
    border-color: #2563eb;
}

.typing {
    background: #f1f5f9;
    padding: 6px 14px;
    border-radius: 20px;
```

```
    display: inline-flex;
    gap: 4px;
}

.typing span {
    width: 6px;
    height: 6px;
    background: #94a3b8;
    border-radius: 50%;
    animation: blink 1.2s infinite;
}

@keyframes blink {

    0%,
    100% {
        opacity: 0.3
    }

    50% {
        opacity: 1
    }
}

.input-wrapper {
    padding: 16px 20px 24px;
    border-top: 1px solid #edf2f7;
}

.input-container {
    max-width: 700px;
    margin: 0 auto;
    display: flex;
    gap: 10px;
}

textarea {
    flex: 1;
    background: #f8fafc;
    border: 1px solid #e2e8f0;
    border-radius: 28px;
    padding: 10px 18px;
    font-family: inherit;
    resize: none;
    outline: none;
    font-size: 0.9rem;
}

textarea:focus {
```

```
    border-color: #2563eb;
}

button.send {
    background: #2563eb;
    border: none;
    width: 40px;
    height: 40px;
    border-radius: 40px;
    color: white;
    cursor: pointer;
}

.footer {
    text-align: center;
    font-size: 0.7rem;
    color: #94a3b8;
    margin-top: 8px;
}
```

requirements.txt

```
installation

pip install flask
pip install requests
```

app.py

```
from flask import Flask, render_template, request, jsonify
import requests
import webbrowser
from threading import Timer

app = Flask(__name__)

api = "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"

API_KEY = api

def ask_groq(question):

    url = "https://api.groq.com/openai/v1/chat/completions"

    headers = {
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json"
    }

    data = {
        "model": "llama-3.1-8b-instant",
        "messages": [
            {"role": "system", "content": "Tu es Hope Assistant, assistant étudiant
intelligent."},
            {"role": "user", "content": question}
        ],
        "temperature": 0.7
    }

    response = requests.post(url, headers=headers, json=data)

    return response.json()["choices"][0]["message"]["content"]


@app.route("/")
def home():
    return render_template("index.html")


@app.route("/chat", methods=["POST"])
def chat():
    question = request.json["question"]

    try:
```

```
        reponse = ask_groq(question)
    except Exception as e:
        reponse = f"Erreur : {str(e)}"

    return jsonify({"reponse": reponse})


# 🔥 OUVRIR AUTOMATIQUEMENT LE NAVIGATEUR
def open_browser():
    webbrowser.open("http://127.0.0.1:5000")


if __name__ == "__main__":
    Timer(1, open_browser).start()
    app.run(debug=True)
```